ИТМО

Analysis & Development of Algorithms

VII

Anton Kovantsev
https://t.me/pop_antonij

Problem statement

How to message human alphabet via wires?

Easy way:

A=1, B=10, C=11, D=100, E=101 … - how to separate?

A=00001, B=00010, C=00011, D=00100 ... - heavy.

How to make this message shorter and faster?

How to avoid mistakes during transmission?

Besides, how to embed the categorical data for ML?

Baudot code (ITA1)

Émile Baudot in the 1870s. - ASSCII - UTF



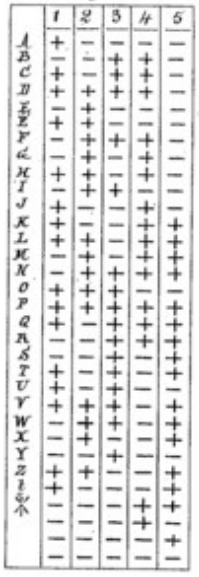
(No Model.) 11 Sheets—Sheet 6.

J. M. E. BAUDOT.

PRINTING TELEGRAPH.

No. 388,244. Patented Aug. 21, 1888.

Fig. 24.



INVENTOR:

Jean Maurice Émile Baudot



ASCII Code Chart

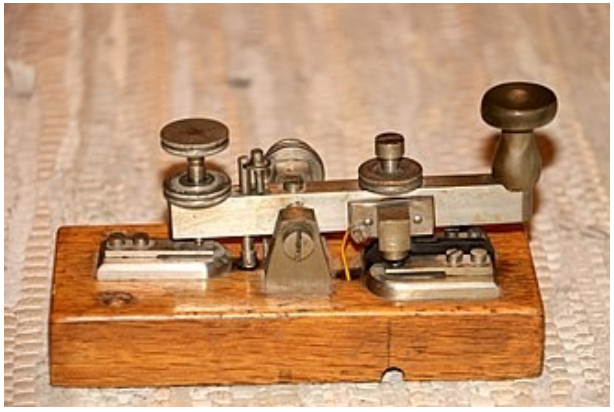
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

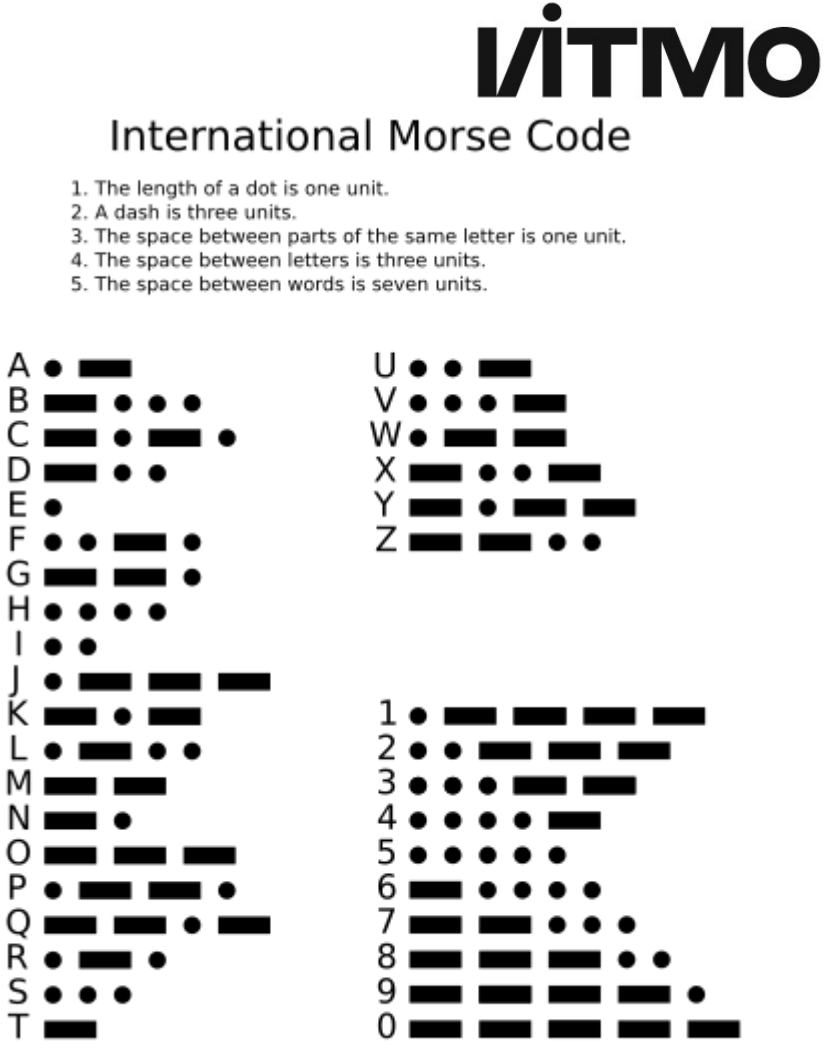
Samuel Morse (~1837)

Morse code is a telecommunications method which encodes text characters as standardized sequences of two different signal durations: *dits* and *dahs* (dots and dashes).

To increase the efficiency of transmission, Morse code was originally designed so that the **duration of each symbol is approximately inverse to the frequency** of occurrence of the character that it represents in text of the English language.

Trouble: it's not binary: the letters of a word are separated by a space of duration equal to **three dits**, and words are separated by a space equal to **seven dits**.





Huffman (1952)

Huffman code is a particular type of **optimal prefix code** that is commonly used for lossless data compression.

Formalized description

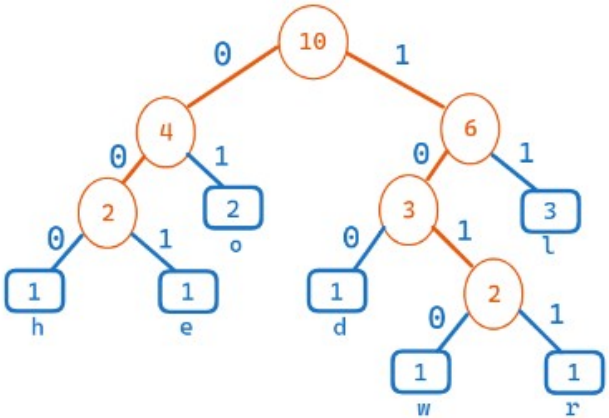
Input: Alphabet $A = (a_1 , a_2 , \ldots , a_n)$, which is the symbol alphabet of size n.

Tuple $W = (w_1 , w_2 , \ldots , w_n)$, which is the tuple of the (positive) symbol weights (usually proportional to probabilities), i.e. $w_i = \mathrm{weight}(a_i)$, $i \in \{ 1 , 2 , \ldots , \mathrm{n} \}$.

Output: Code $C(W) = (c_1 , c_2 , \ldots , c_n)$, which is the tuple of (binary) codewords, where c_i is the codeword for a_i , $i \in \{ 1 , 2 , \ldots , \mathrm{n} \}$.

Goal: $L(C(W)) = \sum_{i=1}^{n} w_i \, lehgth(c_i)$ the weighted path length of code C.

Condition: $L(C(W)) \leq L(T(W))$ for any code $T(W)$.

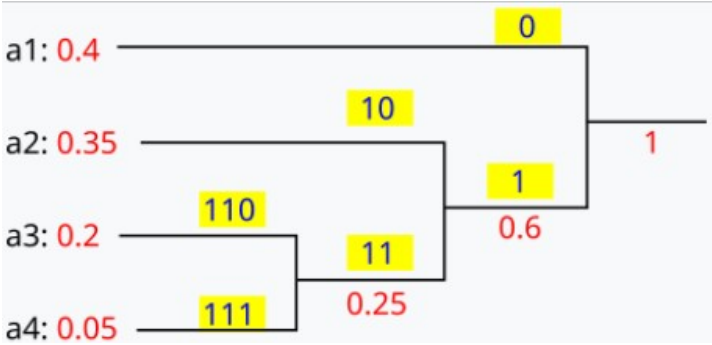


Character		Codes
h	——	000
e	——	001
l	——	11
o	——	01
w	——	1010
r	——	1011
d	——	100

Huffman algorithm

Compression:
The simplest construction algorithm uses a priority queue where the node with lowest probability is given highest priority:

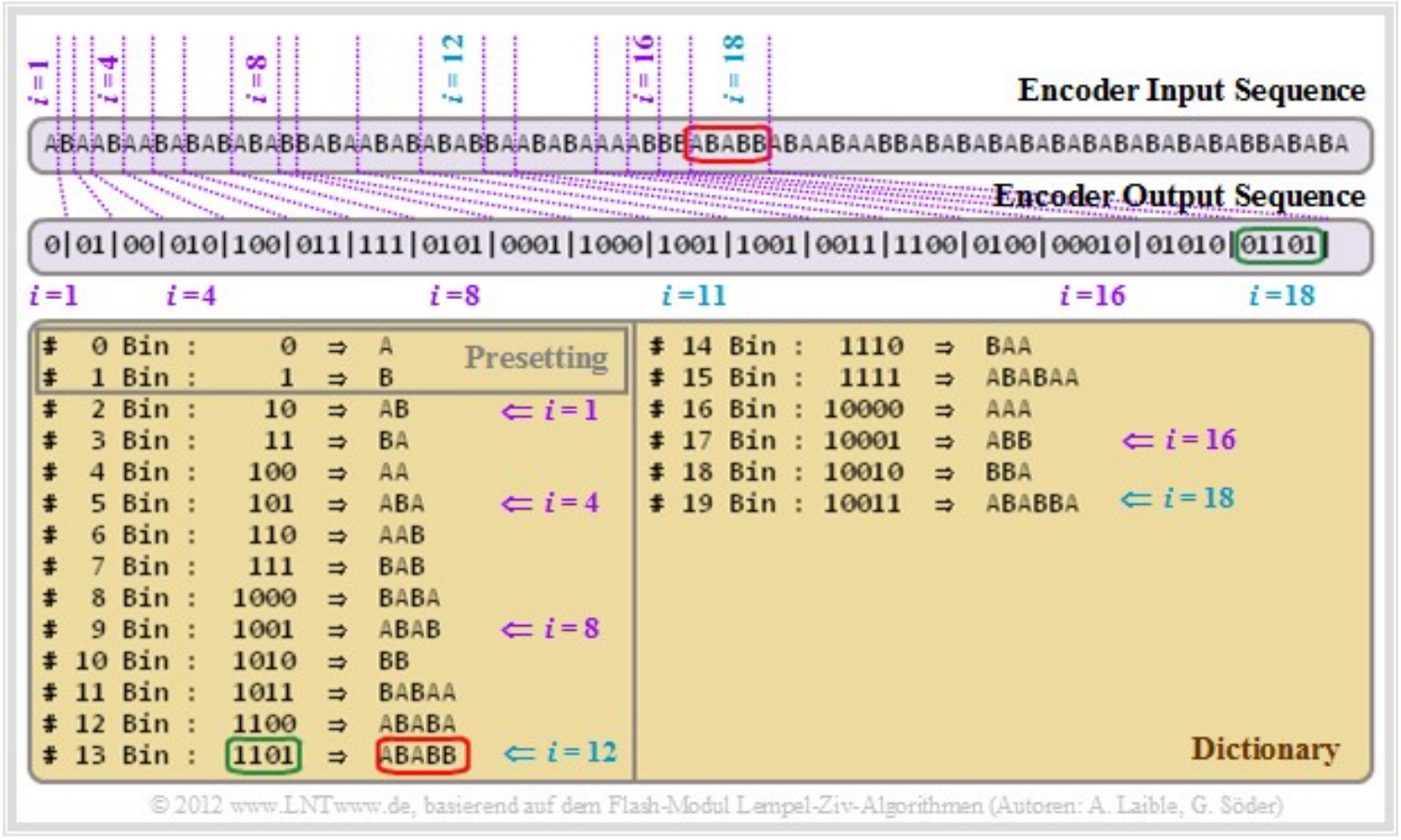
1) Create a leaf node for each symbol and add it to the priority queue.
2) While there is more than one node in the queue:
   Remove the two nodes of highest priority (lowest probability) from the queue
   Create a new internal node with these two nodes as children and with probability equal to the sum of the two nodes' probabilities.
   Add the new node to the queue.
3) The remaining node is the root node and the tree is complete.



Decompression:
The Huffman tree must be somehow reconstructed.
Simply: prepend the Huffman tree, bit by bit, to the output stream.
For example, assuming that the value of 0 represents a parent node and 1 a leaf node, whenever the latter is encountered the tree building routine simply reads the next 8 bits to determine the character value of that particular leaf.
Code table: 0 – a1, 10 – a2, 110 – a3, 111 – a4
Complexity: O(N logN) (Dynamic versions of the algorithm (Vitter) build the tree in a single pass, without requiring a preliminary scan of the file. They are faster).

Lempel-Ziv–Welch (1984)

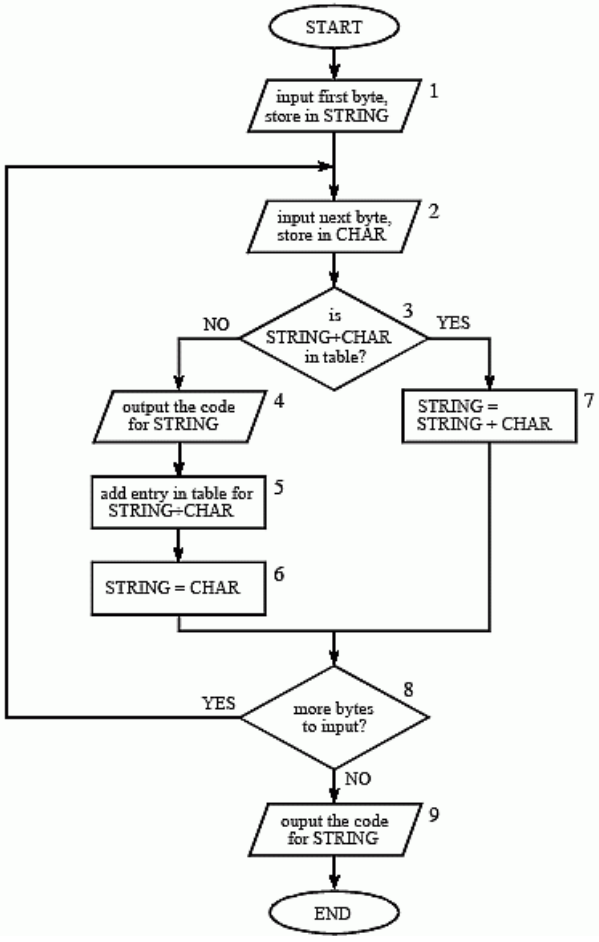
Lempel–Ziv–Welch (LZW) is a universal lossless data compression algorithm.



Lempel-Ziv–Welch (1984)

Encoding:
A high-level view of the encoding algorithm:
1) Initialize the dictionary to contain all strings of length one.
2) Find the longest string **W** in the dictionary that matches the current input.
3) Emit the dictionary index for **W** to output and remove **W** from the input.
4) Add **W** followed by the next symbol in the input to the dictionary.
5) Go to Step 2.

Decoding:
A high-level view of the decoding algorithm:
1) Initialize the dictionary to contain all strings of length one.
2) Read the next encoded symbol: Is it encoded in the dictionary?
    Yes:
        Emit the corresponding string **W** to output.
        Concatenate the previous string emitted to output with the first symbol of **W**. Add this to the dictionary.
    No:
        Concatenate the previous string emitted to output with its first symbol. Call this string **V**.
        Add **V** to the dictionary and emit **V** to output.
3) Repeat Step 2 until end of input string



Error-correcting codes

The central idea is that the sender encodes the message in a **redundant way**, most often by using an error correction code or error correcting code (ECC).[4][5] The redundancy allows the receiver not only to detect errors that may occur anywhere in the message, but often to correct a limited number of errors.

A simplistic example of ECC is to transmit each data bit 3 times, which is known as a (3,1) repetition code.

Triplet received	Interpreted as
000	0 (error-free)
001	0
010	0
100	0
111	1 (error-free)
110	1
101	1
011	1

No one doctor would dare to write a prescription if aspirin and strychnine differed by one letter.

Hamming (1950)

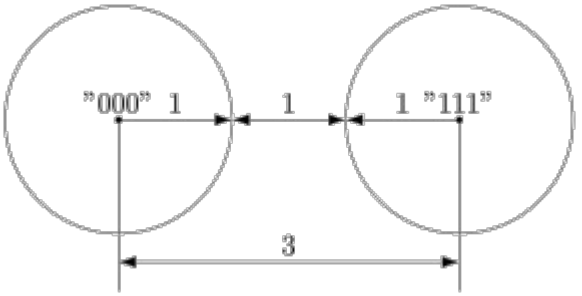
Linear error-correcting code that encodes four bits of data into seven bits by adding three parity bits.
The goal of the Hamming codes is to create a set of parity bits that overlap so that a single-bit error in a data bit or a parity bit can be detected and corrected.

Hamming distance between two strings or vectors of equal length is the number of positions at which the corresponding symbols are different.

1. $d(x, y) \geqslant 0, \quad d(x, y) = 0 \Leftrightarrow x = y;$
2. $d(x, y) = d(y, x);$
3. $d(x, z) + d(z, y) \geqslant d(x, y).$

The number if corrupted bits:

$$k \leqslant \frac{min(d) - 1}{2}$$



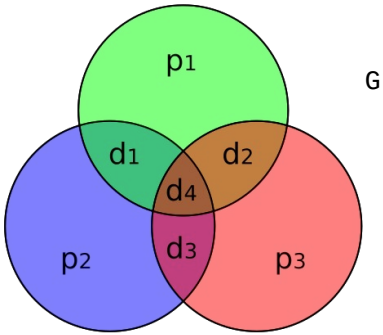
The **main problem** in constructing this kind of code is to arrange the code words so that they are as far apart as possible and there are as many of them as possible.

Hamming (1950)

Hamming(7,4) is a linear error-correcting code that encodes four bits of data into seven bits by adding three parity bits. It can correct any single-bit error

Hamming codes can be computed in linear algebra terms through matrices because Hamming codes are linear codes. For the purposes of Hamming codes, two Hamming matrices can be defined: the code **generator matrix G** and the **parity-check matrix H**:

$$\mathbf{G^T} := \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad \mathbf{H} := \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$




```
      d4 d3 d2 p3 d1 p2 p1
   [  1  0  0  1  0  1  1  ] d4
G = [  0  1  0  1  0  1  0  ] d3
   [  0  0  1  1  0  0  1  ] d2
   [  0  0  0  0  1  1  1  ] d1
```

Correction:

$$\mathbf{z} = \mathbf{Hr} = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

Decoding:

$$\mathbf{R} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Matrix operation on Galois field Gf(2) (XOR)

Galois field is a field that contains a finite number of elements.

$$\bar{x} = Gv^T - encoding\,;\, x - corrupted\,\bar{x}$$
$$e = x - v\ but\ \ >(2) : v = x + e\,;\, x = v + e$$
$$s(x) = Hx^T \neq 0 \Rightarrow error\ occurred\,;\, Hv^T = 0$$
$$Hx^T = H(v+e)^T = He^T = s(e)$$

Hamming (1950)

Encoding:

$$(i_1 \quad i_2 \quad i_3 \quad i_4) \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix} = (i_1 \quad i_2 \quad i_3 \quad i_4 \quad r_1 \quad r_2 \quad r_3)$$

$$\begin{aligned} r_1 &= i_1 \oplus i_2 \oplus i_3, \\ r_2 &= i_2 \oplus i_3 \oplus i_4, \\ r_3 &= i_1 \oplus i_2 \oplus i_4. \end{aligned}$$

Correction:

$$(i_1 \quad i_2 \quad i_3 \quad i_4 \quad r_1 \quad r_2 \quad r_3) \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = (S_1 \quad S_2 \quad S_3)$$

Let (S1, S2, S3) = (1,0,1) => 5-th bit is corrupted

$$\begin{aligned} S_1 &= r_1 \oplus i_1 \oplus i_2 \oplus i_3, \\ S_2 &= r_2 \oplus i_2 \oplus i_3 \oplus i_4, \\ S_3 &= r_3 \oplus i_1 \oplus i_2 \oplus i_4. \end{aligned}$$

$$\begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ \overline{1} \\ 1 \\ 1 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ \underline{0} \\ 1 \\ 1 \end{pmatrix}$$

Decoding:

$$\mathbf{p_r} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

Categorical data embedding

1) Ordinal Encoding
The method is used if the feature has a natural ordered connection.
Examples:
0 - School, 1 - College, 2 - University - good;
0 - dead, 1 - alive (binary features) - good;
0 - BMW, 1 - Audi, 2 - Volkswagen, 3 - Lada - not good. False depending.
1 - January, 2 - February, … 12 - December - usual, but also not good. (from December to January is further than from January to February)

2) One-Hot Encoding (dummy)

	Car_model
0	Toyota
1	Lexus
2	BMW
3	Toyota

→

Car_model_BMW	Car_model_Lexus	Car_model_Toyota
0.0	0.0	1.0
0.0	1.0	0.0
1.0	0.0	0.0
0.0	0.0	1.0

Trouble: Multicollinearity occurs. **WHY?** This can have a negative impact on the quality of the model. To solve the problem, you need to... WHAT?
Another problem is the dimension (o(N)).

Categorical data embedding

3) Binary Encoding
0 - BMW →00
1 - Audi →01
2 - Volkswagen →10
3 - Lada →11
The interpretability of binary features individually is lost.

Dimensionality (o($\log_2 N$)).

4) Contrast Encoding
Encoding divides a feature into levels (each containing only values belonging to one category). Then, some statistics are calculated for each level.

Helmert Encoder — compares the average for a level with the average for all subsequent levels. More suitable for nominal variables.
N features are encoded by a matrix (NxN-1), where the main diagonal of the matrix and above it contains ones with a minus sign, and immediately below the main diagonal there is a serial number of the value and below it zeros.
But: 1) such encoding, as is known, introduces order, which may not be in the data. 2) the values are not normalized and may contribute differently to the vector representations of objects.

	brand	0_0	0_1	0_2	0_3	0_4	0_5
0	BMW	-1.0	-1.0	-1.0	-1.0	-1.0	-1.0
1	Mercedes	1.0	-1.0	-1.0	-1.0	-1.0	-1.0
2	Nissan	0.0	2.0	-1.0	-1.0	-1.0	-1.0
3	Infinity	0.0	0.0	3.0	-1.0	-1.0	-1.0
4	Audi	0.0	0.0	0.0	4.0	-1.0	-1.0
5	Volvo	0.0	0.0	0.0	0.0	5.0	-1.0
6	Skoda	0.0	0.0	0.0	0.0	0.0	6.0

Categorical data embedding

4) Contrast Encoding
Backward-Difference Encoder
Encodes categorical data using a similar scheme as Helmert Encoder, i.e. with a difference before and after the main diagonal.
In the example N=4, k=N-1=3

	brand	brand_0	brand_1	brand_2	brand_3	brand_4	brand_5
0	BMW	-0.857143	-0.714286	-0.571429	-0.428571	-0.285714	-0.142857
1	Mercedes	0.142857	-0.714286	-0.571429	-0.428571	-0.285714	-0.142857
2	Nissan	0.142857	0.285714	-0.571429	-0.428571	-0.285714	-0.142857
3	Infinity	0.142857	0.285714	0.428571	-0.428571	-0.285714	-0.142857
4	Audi	0.142857	0.285714	0.428571	0.571429	-0.285714	-0.142857
5	Volvo	0.142857	0.285714	0.428571	0.571429	0.714286	-0.142857
6	Skoda	0.142857	0.285714	0.428571	0.571429	0.714286	0.857143

	Dummy_1	Dummy_2	Dummy_3
a	$-\frac{k-1}{k}$	$-\frac{k-2}{k}$	$-\frac{k-3}{k}$
b	$\frac{1}{k}$	$-\frac{k-2}{k}$	$-\frac{k-3}{k}$
c	$\frac{1}{k}$	$\frac{2}{k}$	$-\frac{k-3}{k}$
d	$\frac{1}{k}$	$\frac{2}{k}$	$\frac{3}{k}$

The dimension is still o(N), there is still no interpretability.

Categorical data embedding

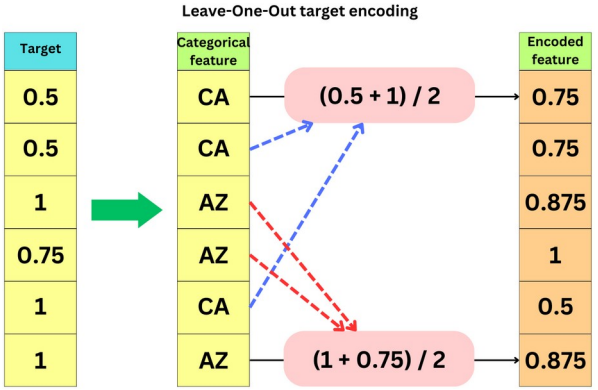
5) Target Encoding

To encode categorical features, a target label is used. (*This is done in the gradient boosting model over decision trees CatBoost*)

- for a regression problem, it uses the mean value of the target variable for a given value of the categorical feature.
- for a binary classification problem, it uses the probability of a single class for a given value of the categorical feature.
- for multi-class classification, it uses the proportion of instances with each value that is lower for each of the classes.

Leave-One-Out Encoder

The encoding of a specific object of the training sample is performed using the average (probability of a single class) without taking into account the value of this object. For the test sample, it is no different from the Target Encoder.



Categorical data embedding

5) Target Encoding

James-Stein Encoder
We code the **weighted average** between the values for a given value of a categorical feature and the value for the entire sample. **Important:** from the formulas for weight B, we can say that this encoder and its optimal parameter B are defined correctly and uniquely in the case when the target label is distributed normally.

$$JS_i = (1 - B_i)\, mean(y_i) + B_i\, mean(y)$$

$$B_i = \frac{var(y_i)}{var(y_i) + var(y)}$$

$$var(y_i) = \mathrm{E}\left[(y_i - \mathrm{E}(y_i))^2\right] \quad \text{regression}$$

$$var(y_i) = \frac{p_i(1 - p_i)}{N_i} \quad \text{classification}$$

• **Handling high cardinality:** It works effectively with features that have a very large number of unique values (e.g., cities, postal codes, IDs) without inflating the dimensionality of the feature space, unlike One-Hot Encoding.
• **Automatic regularization:** Thanks to the James-Stein estimator, rare or unreliable categories do not create significant outliers in the data.
• **Boundedness:** The weight B_i falls within the range from 0 to 1.

Categorical data embedding

6) Vectorising

```
БАЛТ.БЕР.Сел.ПО-ЦАР.ф.т/о.в мас.дым.400г
Сельдь БАЛТИЙСКИЙ БЕРЕГ По-царски атлантическая филе-кусочки в масле с ароматом дыма
(Россия) 400г
Сельдь БАЛТИЙСКИЙ БЕРЕГ По-царски филе-кусочки в масле (Россия) 400г
Сельдь филе-кусочки с/с в масле с укропом по-царски Балтийский берег 400г
```

Doc2Vec generates a vector representation for the entire document or group of words. With this model we can find the relationship between different documents.

Embeddings by the algorithm are features for clustering.

A cluster is something that unites identical entities with different names.

The centroid of a cluster is a vector representation (embedding) of this entity.

Categorical data embedding

6) Vectorising

- Coding based on natural language processing techniques.

Each object is a string of a certain type.

Var1Name	Var2Name	…
Value1	Value2	…
…	…	…

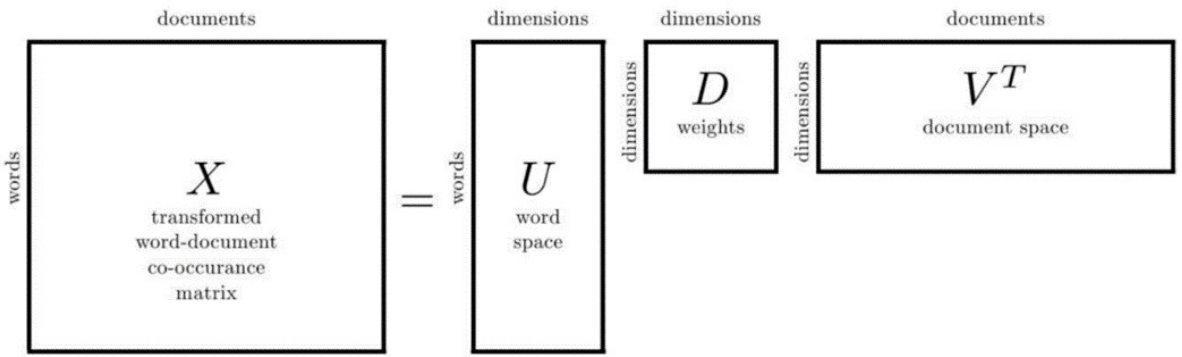
“Var1Name_Value1 Var2Name_Value2 …”

The resulting lines are texts, they have thematic profiles.

LSA (Latent Semantic Analysis) - matrix singular decomposition.

where k is the number of topics, that is, the dimension of our vectors.

$$X = UDV^T \approx U_k D_k V_k^T$$



V_k D_k — object embeddings.

Categorical data embedding

6) Vectorising

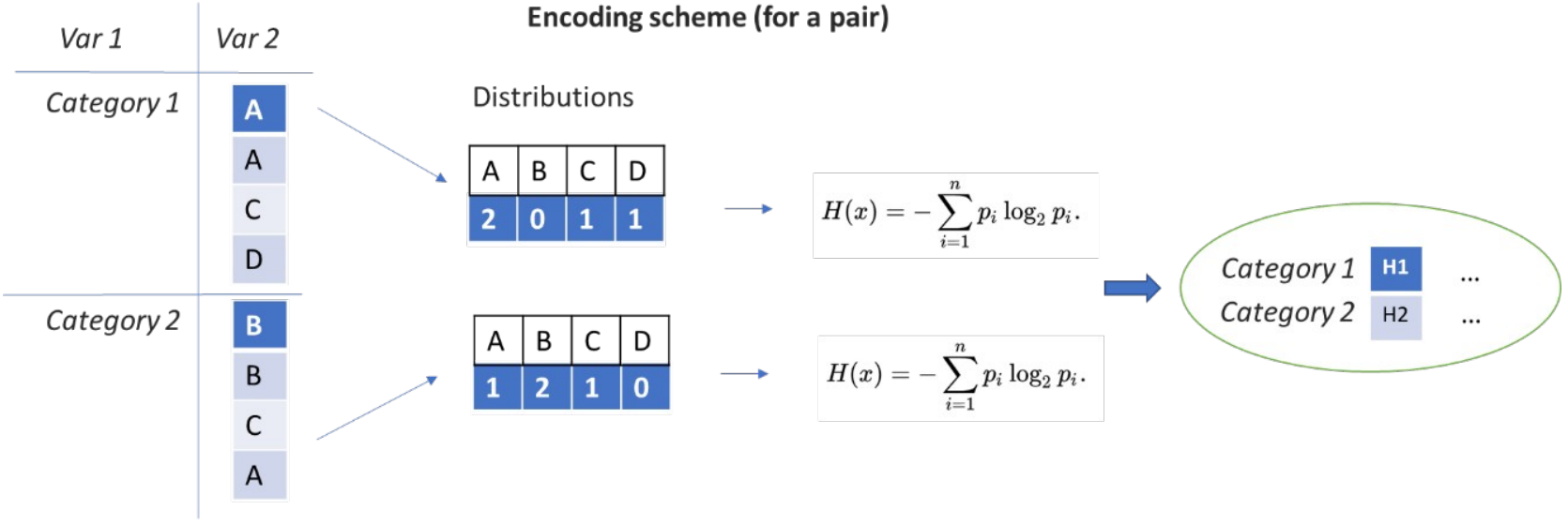
- Coding using elements of probability theory.

Group train by the values of each categorical variable (group_key).

For each of the other variables (passive_key) by the categories group_key, we construct distributions by values, that is, how many times the values of group_key - passive_key were found together.

We calculate entropies by conditional distributions. After combining the results for each group_key, we obtain vectors consisting of the entropies of the distributions of different passive_key.

The dimension now depends on the number of categorical variables: D=N*(N-1) → $o(N^2)$





Антон Кованцев
pop.antonij@gmail.com
https://t.me/pop_antonij